

PHP

Dateizugriff, Dynamische Webseiten

Datenhaltung meist über Datenbanken

- Dennoch manchmal Zugriff auf Dateien notwendig

```
resource fopen($filename, $mode)
```

- > Öffnet Datei \$filename
- > \$mode gibt den Zugriffstypen an: 'r' zum Lesen, 'w' zum Schreiben, 'a' zum Anhängen
- > Tritt ein Fehler auf wird false zurückgegeben

```
bool fclose($handle)
```

- > Schließt den Dateizeiger (eine resource)
- > Rückgabewert true, wenn das Schließen funktioniert hat

```
string fread($handle, $length)
```

- > Liest von dem Dateizeiger bis zu \$length Bytes

```
int fwrite($handle, $string)
```

- > Schreibt den Inhalt von \$string in die Datei, auf die der Dateizeiger zeigt
- > Liefert Anzahl der geschriebenen Bytes zurück oder FALSE im Fehlerfall

```
bool file_exists($filename)
```

- > Gibt TRUE zurück, wenn die Datei oder das Verzeichnis existiert

```
int filesize($filename)
```

- > Liefert die Größe der übergebenen Datei zurück.

Vereinfachter Zugriff auf Dateien:

```
string file_get_contents($filename)
```

- > Liefert den Inhalt der Datei \$filename zurück

```
int file_put_contents($filename, $data)
```

- > Schreibt \$data in die Datei mit dem Namen \$filename
- > Wenn die Datei nicht existiert, wird sie erstellt

Beispiel

```
<?php
    $fileName      = 'title_form.html';
    $fileContent = file_get_contents($fileName);

    echo $fileContent; // Ausgabe des Dateiinhaltes
                       // (auch mit HTML-Tags)

    $fileContent = htmlspecialchars($fileContent);
    echo $fileContent; // Ausgabe mit „entschärften“
                       // HTML-Tags
```

?>

& (ampersand)	&
" (double quote)	"
' (single quote)	'
< (less than)	<
> (greater than)	>

Dynamische Webseiten

Dynamische Webseiten

Statische Webseiten

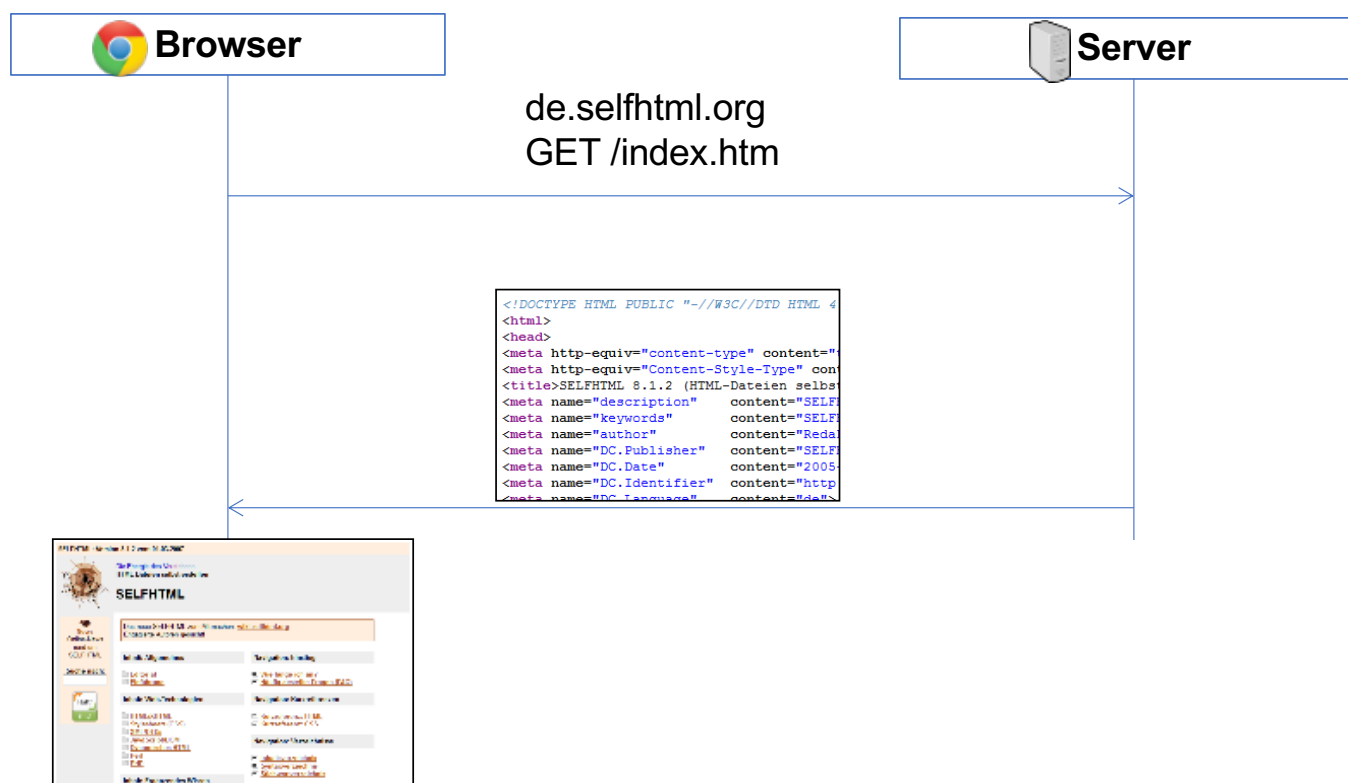
- HTML/CSS-Dateien auf dem Server
 - > Auslieferung an den Browser
 - > Stellen immer den gleichen Inhalt dar
 - > Beispiel: Webvisitenkarte

Dynamische Webseiten

- Erzeugung der Seite im Moment der Anforderung
 - > Darstellung aktueller Daten möglich
 - > Inhalt abhängig von Aktionen des Benutzers
 - > Beispiel: Suchformular

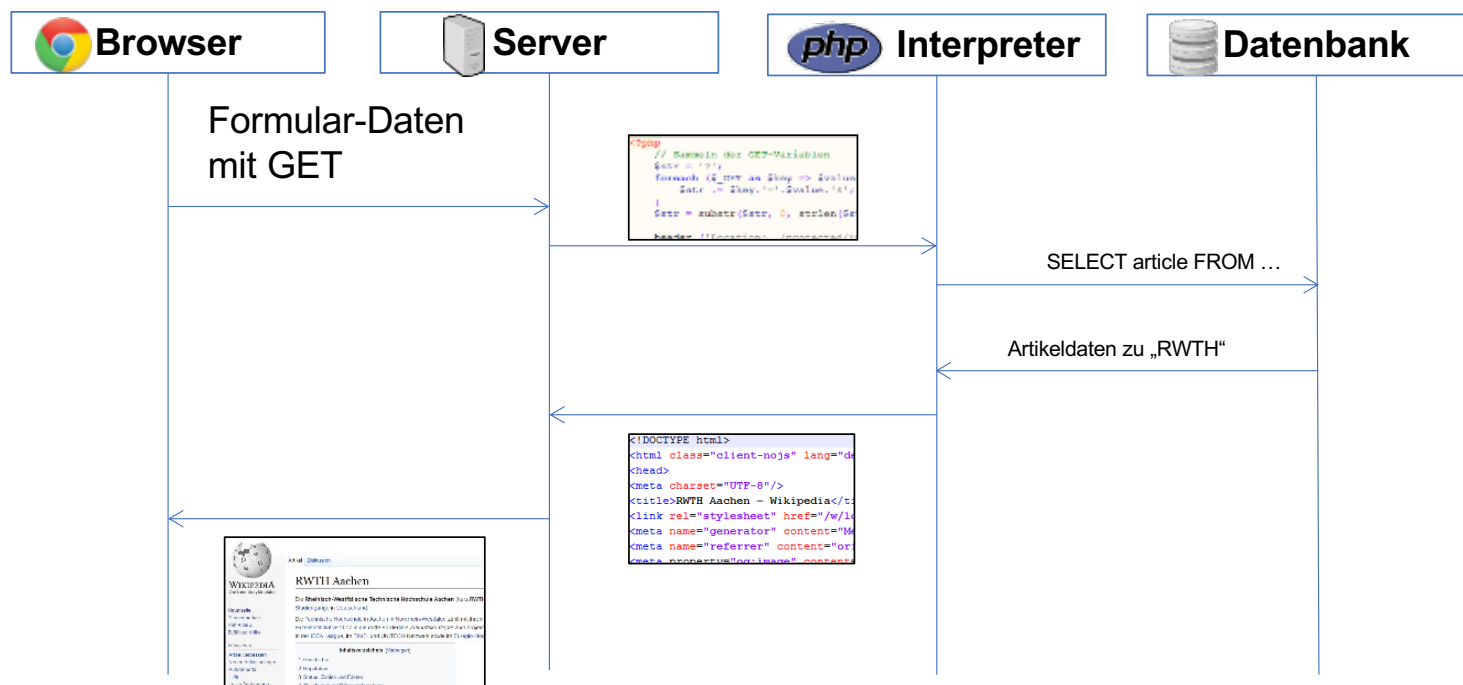
Dynamische Webseiten

Aufruf einer statischen Webseite (vereinfacht)



Dynamische Webseiten

Aufruf einer dynamischen Webseite (vereinfacht)



Beispiel: Login-Formular

```
<form action="do.php?q=login" method="post">
  <input type="text" name="username" />
  <input type="password" name="pw" />
  <input type="submit" value="Login" />
</form>
```

- Action-Anweisung (hier mit relativer URL) gibt an, welche Ressource mit HTTP aufgerufen wird, wenn der Submit-Knopf betätigt wird
- Übermittelt Parameter „username“ und „pw“ per POST-Request
- Auswertung des GET-Parameters (q=login)
 - > Bringt Variabilität in die URL, wir könnten beispielsweise über ein Script mehrere Methoden ansprechen
- Auswertung des Formulars durch PHP
 - > Ist „username“ und „pw“ korrekt?
 - > Evtl. per Datenbankabfrage herausfinden?

GET-Request

- Übertragung der **Parameter über** die **Adresszeile**
- Parameter werden durch das Zeichen ? In der URL eingeleitet
- Kodierung der Parameter gemäß Browser-Kodierung für URLs (%)
- URL inkl. Parameter kann als Bookmark gesichert werden
- Parameter werden in PHP im **globalen Array \$_GET** gespeichert
- Nicht geeignet zur Übertragung großer Datenmengen oder sensibler Daten

- Beispiel: `index.php?id=5&foo=bar`

> Zugriff im PHP-Skript:

```
$_GET['id']    // 5  
$_GET['foo']   // bar
```

Ressourcennamen (Dateinamen)

- RFC3986 empfiehlt zwar die UTF-8 Kodierung, jedoch gibt es hier keinen expliziten Standard. Um etwaige Konflikte zu vermeiden empfiehlt es sich **nur ASCII-Zeichen** als Dateiname zu nutzen
- Andere Zeichen können zwar verwendet werden, sollten aber besser kodiert werden (auch Leerzeichen – Windows!)
- Diverse ASCII-Zeichen wie u.a. %+&=: sind bereits für andere Aufgaben vorgesehen und können ebenfalls so nicht direkt auftauchen
- Für all diese Zeichen gibt es deshalb eine andere Darstellungsform
 - > Nationale Sonderzeichen werden durch ihre hexadezimale Darstellung entsprechend ASCII-Zeichensatz dargestellt. Zur Erkennung dieser Hex-Zeichen wird ein "%" vorangestellt (URL Encoding)
 - Beispiele: ß wird zu %DF,] zu %5D, } zu %7D
 - > Leerzeichen werden zu einem „+“ kodiert („%20“ funktioniert alternativ)
 - > **Browser zeigen heutzutage häufig die dekodierten Werte an**, was dazu führt, dass der Nutzer meist nichts von der Umwandlung merkt und mit den Umlauten arbeiten kann

POST-Request

- Übertragung der Daten im HTTP-Body
 - > Daten sind nicht in der URL sichtbar
 - > Daten werden gemäß Standardkodierung vom Browser verschickt
 - > Im Body steht: `id=5&foo=bar`
 - > Content-Type: `application/x-www-form-urlencoded` (bei reinen Textdaten)
 - > oder sonst Content-Type: `multipart/form-data`

- Zugriff auf POST-Daten in PHP:

```
$_POST['text'] // liefert POST-Parameter text
```

Dynamische Webseiten

Verarbeitung von Formularen

Validierung der Nutzereingaben

- Beschränkungen des Clients können umgangen werden
- Beispiel:

```
<input type="text" name="plz" maxlength="5" />
```
- Parameter kann trotzdem mehr als 5 Zeichen haben
 - > User umgeht aktiv die Beschränkung
 - > Client unterstützt/versteht die Beschränkung nicht
- Serverseitige Prüfung von Nutzerdaten ist zwingend erforderlich
- **„Never trust the user“**



Filter-Funktionen

- Vereinfacht Validierung der Eingaben
- Liefern *null*, *false* oder eine nach *\$filter* gültige Variable zurück
 - > *FILTER_VALIDATE_** validiert Eingaben (alles oder nichts!)
 - > *FILTER_SANITIZE_** korrigiert Eingaben

```
filter_input($type, $variable_name, $filter  
            [, $options])
```

- > Filtert GET/POST-Eingaben
- > \$type gibt die Form der Übertragung an

```
filter_var($variable, $filter [, $options])
```

- > Filtert Variablen oder Werte

Weitere Informationen: <http://php.net/manual/en/filter.filters.php>

filter_input-Beispiel:

```
if (! filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL) ) {  
    echo "E-Mail is not valid";  
} else {  
    echo "E-Mail is valid";  
}
```

Hinweis: Es wird nur auf die syntaktische Korrektheit der eMail geprüft, nicht ob der Benutzer oder der Mail-Server existieren!

filter_input-Beispiel:

```
$heightInCm = filter_input(INPUT_GET, 'height', FILTER_VALIDATE_FLOAT);  
if ($heightInCm === null) {  
    // Parameter 'height' nicht gesetzt  
} else if ($heightInCm === false) {  
    // Parameter 'height' ist kein gültiger Float-Wert  
} else {  
    // Gültiger Float-Wert, $heightInCm ist ein Double!  
}  
  
$range = array ('options' => array ('min_range' => 1, 'max_range' => 100) );  
$choice = filter_input(INPUT_POST, 'choice', FILTER_VALIDATE_INT, $range);  
// $choice ist nun null, false oder ein Integer im Bereich von 1 bis 100  
  
$data = filter_input(INPUT_POST, 'data', FILTER_UNSAFE_RAW);  
// Parameter 'data' so lassen, kann aber optional Zeichen speziell codieren
```